

React.js + Django REST Framework

Full-Stack Learning Path

Concepts First, Then Practice — 5 Modules, 10 Tasks, 1 Capstone Project

- ✓ API Integration
- ✓ Reusable Components
 - ✓ Auth (JWT)
- ✓ Pagination & Filtering
- ✓ Capstone Project

Prepared for onboarding at a company using React + Django REST Framework

Follow modules in order — each one builds on the last

Table of Contents

1. Module 1 — Foundations: REST APIs, DRF Basics & React Fundamentals
2. Module 2 — Connecting React to DRF & the Service Layer Pattern
3. Module 3 — Reusable Components & Custom Hooks
4. Module 4 — Authentication with JWT (DRF SimpleJWT + React)
5. Module 5 — Pagination, Filtering & Search (Production Patterns)
6. Capstone Project — Full-Stack Task Manager (Portfolio-Ready)
7. Appendix — Industry Checklist Before You Ship

How to use this document: each module starts with the **concepts** you need before touching code, followed by hands-on **tasks** that mirror how real teams build features. Do them in order — Module 3 assumes you finished Module 2's project, and so on. Treat each task like a real ticket: create a branch, commit in small logical chunks, and write a short README for what you built.

MODULE 1

Foundations

REST API principles, DRF basics, and React fundamentals you'll use daily

■ CONCEPT — What is a REST API?

REST (Representational State Transfer) is a convention for structuring web APIs around **resources** (e.g. products, users, orders) and standard HTTP verbs: **GET** (read), **POST** (create), **PUT/PATCH** (update), **DELETE** (remove). A resource typically has a collection endpoint (`/api/products/`) and a detail endpoint (`/api/products/5/`). Responses use JSON, and status codes communicate outcome: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Server Error. Every full-stack decision you make will map back to this contract.

■ CONCEPT — Django REST Framework: Serializers & ViewSets

DRF sits on top of Django and turns Python model instances into JSON (and back). A **Serializer** defines which fields are exposed and how they're validated — it's the DRF equivalent of a DTO/schema. A **ViewSet** bundles the standard CRUD logic (list, retrieve, create, update, destroy) for a model, and a **Router** auto-generates the URLs for it. This trio (Model → Serializer → ViewSet + Router) is the standard DRF pattern you'll see in almost every production Django API.

```
backend: models.py / serializers.py / views.py / urls.py
```

```
# models.py
```

```
class Product(models.Model):
```

```
    name = models.CharField(max_length=200)
```

```
    price = models.DecimalField(max_digits=8, decimal_places=2)
```

```
    in_stock = models.BooleanField(default=True)
```

```
# serializers.py
```

```
class ProductSerializer(serializers.ModelSerializer):
```

```
    class Meta:
```

```
        model = Product
```

```
        fields = ['id', 'name', 'price', 'in_stock']
```

```
# views.py
```

```
class ProductViewSet(viewsets.ModelViewSet):
```

```
    queryset = Product.objects.all()
```

```
    serializer_class = ProductSerializer
```

```
# urls.py
```

```
router = DefaultRouter()
```

```
router.register(r'products', ProductViewSet)
```

```
urlpatterns = [path('api/', include(router.urls))]
```

■ CONCEPT — React Fundamentals You'll Actually Use

React apps are trees of **components** — functions that return JSX (HTML-like syntax). Two ideas drive almost everything: **props** (data passed IN to a component from its parent, read-only) and **state** (data a component owns and can change, via `useState`). When state changes, React re-renders. **useEffect** runs side effects — like fetching data — at the right point in a component's lifecycle (e.g. 'when this component first loads' or 'when this ID changes').

TASK 1: Standalone DRF API — Book Catalog

Beginner
~2–3 hrs

Goal: Get comfortable with the Model → Serializer → ViewSet → Router pattern before React enters the picture.

What to build:

- Create a Django project with an app called `catalog`.
- Build a Book model: `title`, `author`, `isbn`, `price`, `published_date`, `is_available`.
- Write a `ModelSerializer` and a `ModelViewSet` for it, wired through a `DefaultRouter`.
- Test all 5 CRUD operations in Postman/Insomnia (or DRF's browsable API).
- Add basic field validation (e.g. `price` must be `> 0`) using a serializer `validate_price` method.

Industry standard to apply: Use Django's app-per-domain structure (don't dump everything in one app). Keep settings in environment variables via `python-decouple` or `django-environ` — never hardcode secrets.

Deliverable: A running DRF API + a Postman collection (exported as JSON) proving all endpoints work.

TASK 2: Plain React Fetch — Read-Only Book List

Beginner
~2 hrs

Goal: Practice `useState` + `useEffect` + conditional rendering before building the service-layer abstraction.

What to build:

- Create a React app (Vite recommended: `npm create vite@latest`).
- Build a `BookList` component that fetches from your Task 1 API using plain `fetch()` inside `useEffect`.
- Handle three UI states explicitly: loading, error, and success-with-data.
- Render each book as a card showing title, author, and price.

Industry standard to apply: Never leave a fetch without error handling — a failed request should never leave the UI stuck on a blank spinner forever.

Deliverable: A React app that lists books from your live DRF API, with visible loading/error states.

MODULE 2

Connecting React to DRF

The service layer pattern — how real teams organize API calls

■ CONCEPT — Why You Never Call `fetch/axios` Directly Inside Components

In a small demo, calling `fetch()` inside a component is fine. In a real codebase with 30+ screens, it becomes unmaintainable — every component reinvents headers, base URLs, and error handling. The industry pattern is a **service layer**: one shared, configured HTTP client (usually `axios`, with interceptors for auth headers and global error handling), and one file per resource exposing plain functions (`getAll`, `getOne`, `create`, `update`, `remove`). Components import these functions and never touch `axios/fetch` directly. This mirrors DRF's own resource-based thinking on the frontend.

```
frontend: src/api/client.js + src/api/books.js

// src/api/client.js
import axios from 'axios';

const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  headers: { 'Content-Type': 'application/json' },
});

export default apiClient;

// src/api/books.js
import apiClient from '../client';

export const booksApi = {
  getAll: () => apiClient.get('/books/'),
  getOne: (id) => apiClient.get(`/books/${id}/`),
  create: (data) => apiClient.post('/books/', data),
  update: (id, data) => apiClient.put(`/books/${id}/`, data),
  remove: (id) => apiClient.delete(`/books/${id}/`),
};
```

■ CONCEPT — CORS — the error every beginner hits

By default, browsers block a React app on `localhost:5173` from calling a Django API on `localhost:8000` — this is CORS (Cross-Origin Resource Sharing). Fix it in Django with `django-cors-headers`, whitelisting your frontend's origin in `CORS_ALLOWED_ORIGINS`. This is a real setting you'll configure in every project.

TASK 3: Full CRUD — Book Manager (Service Layer)

Beginner–Intermediate
~3–4 hrs

Goal: Rebuild Task 2 properly: full CRUD, through a service layer, against your real API.

What to build:

- Install and configure `django-cors-headers` on the backend.
- Build the `axios` client + `booksApi` module as shown above.
- Build an 'Add Book' form and a 'Delete' button per row, both calling `booksApi`.
- After create/update/delete, refresh the list so the UI reflects the backend truth (no stale data).
- Show validation errors from DRF (400 responses) directly under the relevant form field.

Industry standard to apply: Store the API base URL in a `.env` file, never hardcoded — this is how you'll switch between local/staging/production without touching code.

Deliverable: A working CRUD app where all network logic lives in `src/api/`, not in components.

Reusable Components & Custom Hooks

The part of React that actually saves you time on every future project

■ CONCEPT — Presentational vs. Container Components

A **presentational** component only cares about how things look — it receives data and callbacks via props and has no idea where that data came from (a Button, a DataTable, a Card). A **container** component knows how to fetch/manage data and passes it down. This separation is what makes a component genuinely reusable across projects — a DataTable that only knows about 'columns' and 'rows' can render books, orders, or users without modification.

■ CONCEPT — Custom Hooks: Extracting Repeated Logic

Any time you find yourself copy-pasting the same useState + useEffect pattern across components, it belongs in a **custom hook** — a plain function starting with 'use' that itself calls other hooks. The most common one you'll write is useApi or useFetch, which wraps loading/error/data state around any API call, so components just say what to fetch, not how to manage the fetching.

frontend: src/hooks/useApi.js

```
// src/hooks/useApi.js
export function useApi(apiFunc, immediate = true) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(immediate);
  const [error, setError] = useState(null);

  const execute = useCallback(async (...args) => {
    setLoading(true); setError(null);
    try {
      const res = await apiFunc(...args);
      setData(res.data);
      return res.data;
    } catch (err) {
      setError(err.response?.data || err.message);
      throw err;
    } finally { setLoading(false); }
  }, [apiFunc]);

  useEffect(() => { if (immediate) execute(); }, [immediate, execute]);
  return { data, loading, error, refetch: execute };
}
```

TASK 4: Reusable DataTable + Form Components

Intermediate
~3 hrs

Goal: Refactor your Book Manager so the UI is built from generic, reusable pieces instead of one-off JSX.

What to build:

- Build `<DataTable columns={[]} data={[]} onClick />` — fully generic, no 'book' logic inside it.
- Build `<FormField label error ... />` that renders a labeled input and any DRF validation error for that field.
- Extract your fetch logic into `useApi` and use it for the book list.
- Re-render your Book Manager using only these three reusable pieces.

Industry standard to apply: Components take props and emit callbacks — they should never import a specific API module directly (that's the container's job). This is what 'reusable' means in a code review.

Deliverable: The same Book Manager feature, now built from components you could drop into a completely different project.

Authentication with JWT

How real apps handle login, protected routes, and token refresh

■ CONCEPT — JWT Auth Flow (DRF SimpleJWT)

JSON Web Tokens let a stateless API verify who's asking without storing sessions server-side. The flow: (1) user submits credentials to `/api/token/`, (2) DRF SimpleJWT returns an **access token** (short-lived, sent with every request) and a **refresh token** (long-lived, used to get a new access token when it expires), (3) the frontend attaches `Authorization: Bearer <access_token>` to every request via an axios interceptor, (4) on a 401 response, the frontend silently uses the refresh token to get a new access token before retrying.

■ CONCEPT — Protected Routes in React

Not every page should be visible to a logged-out user. The standard pattern is a `<ProtectedRoute>` wrapper component that checks auth state (usually via React Context, e.g. `AuthContext`) and redirects to `/login` if there's no valid token — used alongside React Router's `<Navigate />`.

frontend: axios interceptors for JWT

```
// src/api/client.js (interceptor additions)
apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('access_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

apiClient.interceptors.response.use(
  (res) => res,
  async (error) => {
    if (error.response?.status === 401) {
      // attempt refresh, else redirect to /login
    }
    return Promise.reject(error);
  }
);
```

TASK 5: Add Login & Signup

Intermediate
~3–4 hrs

Goal: Secure your Book Manager so only logged-in users can add/edit/delete books; anyone can view.

What to build:

- Install `django-rest-framework-simplejwt`; expose `/api/token/` and `/api/token/refresh/`.
- Restrict write operations on the ViewSet using DRF permission classes (`IsAuthenticatedOrReadOnly`).
- Build Login and Signup forms in React; store tokens in `localStorage` on success.
- Add the auth interceptors shown above; create an `AuthContext` exposing `user`, `login()`, `logout()`.
- Wrap the 'Add Book' page in a `<ProtectedRoute>`.

Industry standard to apply: Never store passwords or long-lived secrets in `localStorage` — only tokens. Log the user out client-side if a refresh attempt fails.

Deliverable: A Book Manager where write actions require login, with a visible logged-in/out state in the navbar.

MODULE 5

Pagination, Filtering & Search

How production APIs handle real data volumes

■ CONCEPT — DRF Pagination

Returning 10,000 rows in one response is never acceptable. DRF's `PageNumberPagination` wraps responses as `{count, next, previous, results}`. The frontend reads `next/previous` to build Next/Previous controls instead of guessing page counts.

■ CONCEPT — Filtering & Search with `django-filter`

Instead of writing custom query logic per field, `django-filter` lets you declare which fields are filterable directly on the `ViewSet` (`filterset_fields`), and DRF's built-in `SearchFilter` handles free-text search across chosen fields via a `?search=` query param. This is the standard way production DRF APIs expose querying without hand-rolling it.

backend: filtering + search + ordering

```
# views.py
class BookViewSet(viewsets.ModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    filter_backends = [DjangoFilterBackend, filters.SearchFilter, filters.OrderingFilter]
    filterset_fields = ['is_available', 'author']
    search_fields = ['title', 'author']
    ordering_fields = ['price', 'published_date']
```

TASK 6: Paginated, Searchable Book Catalog

Intermediate–Advanced
~3–4 hrs

Goal: Make your catalog behave like a real product listing page.

What to build:

- Enable DRF pagination (page size 10) on the Book endpoint.
- Add `django-filter` for `is_available` and `author`, plus search on title/author.
- Build a search bar and a Next/Previous control in React, driven by URL query params (so a page refresh preserves state).
- Debounce the search input (wait ~400ms after typing stops) before firing a request.

Industry standard to apply: Keep query state in the URL (e.g. `?search=tolkien&page=2`), not just component state — this is what makes a listing page shareable and bookmarkable.

Deliverable: A catalog page with working pagination, search, and at least one filter, backed by real query params.

Full-Stack Task Manager

Combine every concept above into one portfolio-ready project

This is the project to put on your CV/portfolio and to discuss in interviews. It deliberately mirrors what a small internal tool looks like on the job — the same shape as most CRUD-based admin dashboards you'll build professionally.

■ CONCEPT — Scope

A multi-user Task Manager: users sign up/log in, create projects, and add tasks to a project with a status (To Do / In Progress / Done), a priority, and a due date. Users can only see and manage their own projects.

TASK 7: Capstone — Backend

Advanced
~5–6 hrs

Goal: Model a realistic multi-resource, multi-user domain in DRF.

What to build:

- Models: `Project` (owner=FK to `User`, name), `Task` (FK to `Project`, title, status, priority, due_date).
- ViewSets + routers for both, with pagination, filtering by status/priority, and search by title.
- Permission logic: a user can only see/edit `Projects` and `Tasks` they own (override `get_queryset` to filter by `request.user`).
- JWT auth wired in exactly as in Module 4.
- Write at least 5 DRF test cases (e.g. using `APITestCase`) covering the ownership restriction.

Industry standard to apply: Never trust the frontend to enforce permissions — ownership checks belong in the backend `queryset`/permission classes, always.

Deliverable: A tested, authenticated, multi-resource DRF API with proper per-user data isolation.

TASK 8: Capstone — Frontend

Advanced
~6–8 hrs

Goal: Assemble everything from Modules 2–5 into one cohesive app.

What to build:

- Service layer for both projects and tasks resources.
- Reusable `DataTable`, `FormField`, and a new `<StatusBadge status />` component (color-coded by status).
- Auth context + protected routes for the whole dashboard.
- A Kanban-style board view (columns = To Do / In Progress / Done) as a stretch goal, using drag-and-drop (e.g. `@dnd-kit/core`) to update task status via PATCH.
- Search, filter-by-priority, and pagination on the task list, exactly as in Module 5.

Industry standard to apply: Split components into `components/` (reusable), `pages/` (routes), `api/`, `hooks/`, and `context/` — this is the folder structure you'll see in almost every production React app using this stack.

Deliverable: A deployed (or at least fully runnable) multi-page app: login → dashboard → project detail → task board.

TASK 9: Polish — Deployment & README

Intermediate
~2–3 hrs

Goal: Ship it the way a real team would hand off a project.

What to build:

- Add environment-based settings (dev/prod) on the Django side (e.g. DEBUG off in prod, allowed hosts set).
- Deploy the backend (e.g. Railway/Render) and frontend (e.g. Vercel/Netlify); wire the deployed frontend to the deployed API.
- Write a README: setup steps, environment variables required, and a short architecture overview with a diagram of Model → Serializer → ViewSet → React service → component.

Industry standard to apply: A project without a README and clear `.env.example` is not considered 'done' on a real team — always ship both.

Deliverable: A live URL you can share, plus a repo with a README a new teammate could actually onboard from.

TASK 10: Stretch — Code Review Simulation

Advanced
~2 hrs

Goal: Practice the review habits senior engineers will expect from you.

What to build:

- Re-read your own capstone PR as if you were reviewing a teammate's code.
- Check for: components doing too much, missing loading/error states, hardcoded values that should be env vars, and any place the frontend duplicates a check the backend should own.
- Refactor at least 3 things you find, and note them in your README under 'What I'd improve next.'

Industry standard to apply: This self-review habit — reading your own diff before requesting review — is one of the fastest ways to build trust with a new team.

Deliverable: A short 'lessons learned' note alongside your capstone repo.

Appendix — Industry Checklist Before You Ship Any Feature

Area	Checklist
API design	Resource-based URLs, correct HTTP verbs, meaningful status codes, consistent error shape
Security	Secrets in env vars, CORS locked to known origins, permissions enforced server-side, no secrets in localS
Frontend structure	api/ · hooks/ · components/ · pages/ · context/ kept separate; no direct axios calls in components
State & data	Loading/error/empty states handled explicitly; query params reflect list state (search, page, filters)
Git workflow	Feature branches, small focused commits, PR description explains 'why' not just 'what'
Testing	At least basic DRF APITestCase coverage on permissions and core CRUD paths
Docs	README with setup steps, .env.example, and a one-paragraph architecture overview

Suggested pace: 1 module every 3–5 days alongside your normal onboarding work. By the end of the capstone you'll have shipped the same Model → Serializer → ViewSet → service layer → reusable component → protected route pattern your team's real project almost certainly follows.